

AI Agents for Financial Analysis

Key Concepts on Financial Document Analysis with Large Language Models

Michaël Karpe

March 23-24, 2026

michael.karpe@berkeley.edu. All rights reserved.

Table of contents

1. Module 1: Why AI Agents in Finance
2. Module 2: SEC Filings as an Open Alternative Data Source
3. Module 3: RAG and Common Techniques
4. Module 4: Technical & Pricing Challenges
5. Appendix: Gemini API key setup

Module 1: Why AI Agents in Finance

Why Now?

- **Finance AI market:** \$38.36B in 2024, projected \$190.33B by 2030 (Markets and Markets)
- **Gartner:** By 2028, 33% of enterprise software will include agentic AI (up from less than 1% in 2024)
- **Deloitte 2025 CFO Survey:** 47% of finance teams have already deployed at least one AI agent
- **KPMG:** \$50 billion in global spend on agentic AI in 2025 alone
- **McKinsey:** AI could reduce certain cost categories by up to 70% across banking

⇒ *The question is no longer "if" but "how"*

What Is an AI Agent? (vs. Traditional AI vs. GenAI)

- **Traditional AI:** Predefined rules, responds to specific inputs based on past data
- **Generative AI:** Creates new content from patterns, but needs a human prompt every step
- **AI Agent:** Perceives data → Reasons → Takes Actions → Learns & Adapts → Coordinates
 - **Key capability:** Autonomous multi-step planning without human intervention
 - **Memory:** An agent recalls what it did before and adjusts based on outcomes
 - **Example:** Reconcile invoices with purchase orders → an agent handles the full workflow end-to-end

AI Agents vs. Robotic Process Automation

- **RPA (Robotic Process Automation):** Executes fixed, scripted workflows → brittle and rigid
 - **Rule-Based:** If-then logic, no learning or adaptation
- **AI Agents:** Dynamic, context-aware, capable of handling exceptions and novel situations
 - **Finance example:** RPA can route invoices; AI agents can flag anomalies, query missing data, escalate intelligently

Key Differentiator

Agents use LLMs, tools, memory and planning.

Use Cases Across Financial Workflows

- **Trading:** Portfolio optimization, sentiment analysis
- **Risk Management:** Credit scoring, stress testing
- **Compliance:** KYC/AML automation, audit trails
- **Financial Planning & Analysis (FP&A):** Budget variance, forecasting
- **Research:** SEC filing analysis, benchmarking
- **Back Office:** Invoice processing, reconciliation

Use case example

IBM: AI + RPA for journal entries → 90% reduction in cycle time, \$600K annual savings.

The Full AI Agent Pipeline

1. **Ingestion:** Raw financial data from multiple sources (APIs, PDFs, EDGAR, market feeds)
2. **Processing:** Chunking, cleaning, structuring, embedding
3. **Storage:** Vector databases, relational DBs, knowledge graphs
4. **Retrieval:** Semantic search, hybrid retrieval (vector + BM25)
5. **Reasoning:** LLM with context → analyze, synthesize, generate structured output
6. **Orchestration:** Multi-agent coordination, tools, memory, human-in-the-loop
7. **Output:** Reports, alerts, dashboards, automated actions

Module 2: SEC Filings as an Open Alternative Data Source

Financial Data Sources: Taxonomy

- **Structured:** Balance sheets, income statements, market prices, XBRL filings
- **Semi-structured:** SEC EDGAR filings (10-K, 10-Q, 8-K), earnings transcripts
- **Unstructured:** News articles, analyst reports, social media sentiment
- **Real-time:** Stock feeds, options chains, FX rates, macro indicators
- **Alternative:** Satellite data, credit card transactions, web traffic, job postings

Challenge

Most AI value in finance comes from integrating across ALL these data types simultaneously.

Example: SEC EDGAR Filing Types Taxonomy

- **SEC EDGAR:** Authoritative, free, structured public data
- **10-K:** Annual report → full financials, risk factors, business overview
- **10-Q:** Quarterly update → interim financials, material changes
- **8-K:** Current report → material events (earnings, M&A)
- **Form 4:** Insider transactions → executives buying/selling shares
- **13F:** Institutional holdings → fund ownership
- **XBRL tagging:** Programmatic extraction of financial data

Introducing edgartools: Python Library for SEC Data

- `pip install edgartools`
- **Key features:** Company lookup, filing retrieval, XBRL parsing
- **Access financials:** Income statement, balance sheet, cash flow
- **Export:** Markdown, pandas DataFrames, or raw text

Code Example

```
from edgar import Company
company = Company('LBHI')
filings = company.get_filings(form='10-K')
```

Data Quality & Pre-Processing Challenges

- **Document heterogeneity:** Tables, footnotes, exhibits → need different parsers
- **Inconsistent formatting:** Pre-2009 filings are HTML/text, not XBRL
- **Long documents:** 10-K can be 200+ pages (exceeds LLM context)
- **Chunking strategy:** Fixed-size vs. semantic chunking (critical for RAG)
- **Metadata:** Preserve company, date, filing type, section headers

Challenge

Retrieval quality is directly proportional to chunking quality.

Embedding: Turning Text Into Vectors

- Converts text into high-dimensional vectors capturing semantic meaning
- **Similar meaning** → **similar vectors** (cosine similarity)
- **Key models:** OpenAI text-embedding-3, Google gemini-embedding-001
- **Financial domain:** FinBERT, FinLang → pre-trained on financial text
- **Dimensions:** generally 768 to 3072 depending on model

Important

Use the SAME embedding model at ingestion and at query time.

Module 3: RAG and Common Techniques

What Is RAG and Why Does It Matter?

- **RAG = Retrieval Augmented Generation:** retrieve context → augment prompt
- **Without RAG:** GPT-4 Turbo answers only ~19% of financial questions correctly
- **With RAG:** Multi-agent RAG systems achieve 56% accuracy on FinanceBench
- **Solves:** Hallucination, outdated knowledge, missing company-specific data

Finance Example

"What are the liquidity risks in the latest 10-K?"

Example: Qdrant Vector Database

- Open-source, production-grade vector database
- **Performance:** built in Rust, 3ms response for 1M OpenAI embeddings
- **Hybrid search:** Combines dense vector search + sparse (BM25)
- **Metadata support:** Store ticker, date, section alongside vectors

Example Code

```
from qdrant_client import QdrantClient
client = QdrantClient('localhost', port=6333)
client.create_collection(collection_name='filings', ...)
```

Chunking Strategies

- **Some chunking strategies:**
 - **Fixed-size:** Fast but splits mid-sentence/mid-table
 - **Semantic:** Splits at paragraph/section boundaries
 - **Hierarchical:** Parent-child chunks → summary then detail
- **Best practices:**
 - **Follow document structure:** Chunk by section is often best
 - **Overlap:** 10–20% to prevent losing context at boundaries
 - **Size:** 512–1024 tokens is typical

Retrieval Strategies: Beyond Simple Similarity Search

- **Naive RAG:** Query $\rightarrow$ top-K cosine similarity (limited)
- **Advanced techniques:**
 - **Hybrid retrieval:** Dense (semantic) + sparse (keyword)
 - **MMR (Maximal Marginal Relevance):** Reduces redundancy
 - **Reranking:** Use a reranker to reorder results of hybrid retrieval
 - **Query expansion:** Rephrase query multiple times (HyDE)
 - **Filtered retrieval:** Restrict search (e.g., only Q1 2008)

Agentic RAG: RAG as an Agent Tool

- **Traditional RAG:** query → retrieve → generate (linear)
- **Agentic RAG:** Agent DECIDES when/how to retrieve, can retry
 - **LangGraph:** Framework for building stateful agentic workflows
 - **Example:** Agent sees context gap → reformulates query → retries
 - **Benefit:** "Compare leverage ratios across quarters 2006-2008"

Module 4: Technical & Pricing Challenges

- **Proprietary LLMs:**
 - **Google Gemini 3.1 Pro:** 1M token context; multimodal; \$2–\$4/M input
 - **Anthropic Claude Opus 4.6:** 1M context; strong structured output; \$5/M input
 - **OpenAI GPT-5.4:** 1M context; strong coding + finance; \$2.5/M input
- **Open-source LLMs:** fully self-hosted, data privacy. Mid-size models:
 - **Qwen 3.5 27B:** 262k context, vision, tools, thinking, SOTA in March 2026, China
 - **Gemma 3 27B:** 128k context, vision, tools, SOTA in March 2025, USA
 - **Mistral Small 3.2 24B:** 128k context, vision, France

Challenge

10-K filings can be 500K+ tokens; proprietary models preferred.

LLM Pricing Tiers: Choosing the Right Model

- **Model Tiers:**

- **Tier 1 → Premium:** Claude Opus, GPT-5, Gemini Pro (Complex synthesis)
- **Tier 2 → Mid-range:** Claude Sonnet, GPT-5-mini, Gemini Flash (Analysis)
- **Tier 3 → Economy:** Claude Haiku, GPT-5-nano, Gemini Flash Lite (Classification)

- **Optimization Strategies:**

- **Prompt caching:** Up to 90% tokens discount (Claude)
- **Batch processing:** 50% discount (OpenAI Batch API)
- **Model routing:** Classifier to route simple queries

Context Window Challenges

- **10-K Report:** 80K–150K tokens (~100+ pages)
- **GPT-4o (128K):** Full 10-K barely fits (expensive)
- **Gemini 2.5 Pro (1M):** Multiple years of filings fit
- **Claude (200K):** Suitable for single large filings
- **Problem:** 'Lost in the middle' (loss of focus on long docs)

Challenge

Minimizing cost while maintaining high quality → RAG for focused retrieval + long context for final synthesis.

Hallucination, Reliability & Validation

- **Mitigation 1:** Always ground in retrieved text + citations
- **Mitigation 2:** Temperature = 0 for factual extraction
- **Mitigation 3:** Multi-agent validation (second agent checks)
- **Mitigation 4:** Structured output schemas (JSON)
- **Mitigation 5:** Confidence scoring

Challenge

Never use AI output directly without validation against sources.

Depending on the country and the industry. A few examples:

- **EU AI Act:** Some AI in finance applications are "high-risk"
- **Model Risk Management:** Banking guidance
- **GDPR/CCPA:** Processing personal financial data
- **Explainability:** ECOA / Fair Lending requirements
- **DORA:** Cloud vs. on-premise deployment

Appendix: Gemini API key setup

Gemini API key setup

To get the most out of the workshop, it is recommended to:

- know how to use Google Colaboratory (<https://colab.research.google.com>)
- if you don't have one, create a Google Cloud project linked to a billing account. You can "Start free trial" at <https://console.cloud.google.com> to complete this step if you have not used the free trial already. Credit card is requested only for identity verification purposes.
- have a free Google AI Studio API key created under a Google Cloud project linked to a billing account (<https://aistudio.google.com/app/api-keys>). Make sure the API key "Quota Tier" is set to "Free trial". You do not need to click on "Activate billing".